Computación en nube y virtualización de redes y servicios (CNVR)

Arquitectura de redes y servicios en la nube (ARSN)

Curso 2026-27

Práctica 1.4
Servicios de orquestación en OpenStack y Kubernetes: Terraform

Versión del 21 de octubre de 2026

Objetivo

El objetivo de la práctica es comprender el funcionamiento de los servicios de orquestación que ofrece Terraform, experimentando sus funcionalidades más básicas mediante el despliegue de escenarios sencillos sobre los entornos virtuales de prácticas de OpenStack y Kubernetes.

1 - Introducción

Terraform es un *software* de infraestructura como código (*Infrastructure as Code* o IaC) desarrollado por HashiCorp. Permite a los usuarios definir y configurar la infraestructura de un centro de datos en un lenguaje de alto nivel, siguiendo los principios de automatización de recursos según las metodologías de Integración y Despliegue Continuos (*Continuous Integration* / *Continuous Delivery* o CI/CD), generando un plan de ejecución para desplegar los recursos en OpenStack, por ejemplo, u otros proveedores de servicios tales como Amazon Web Service (AWS), Google Cloud, o Microsoft Azure, o Kubernetes. La infraestructura se define utilizando la sintaxis de configuración de HashiCorp denominada HashiCorp Configuration Language (HCL) o, en su defecto, el formato JSON. El formato HCL es más legible, admite comentarios y es el más recomendado para la mayoría de los archivos de configuración de Terraform.

Aunque OpenStack dispone de un sistema de orquestación nativo como Heat y Kubernetes tiene su propio lenguaje declarativo de recursos mediante plantillas en formato YAML, en esta práctica trabajaremos con Terraform, pues es una solución flexible con soporte para múltiples proveedores de servicios, entre ellos OpenStack y Kubernetes, y que dispone de una comunidad de desarrollo activa. Además, el lenguaje HCL permite estructurar la definición de recursos en diferentes módulos, los cuales son reutilizables y permiten gestionar la infraestructura de manera modular y más eficiente. La versión de Terraform para proveer servicios en OpenStack permite especificar escenarios compuestos por todo tipo de recursos que se pueden crear en OpenStack (máquinas virtuales, claves, redes, subredes, etc.), permitiendo su creación y la gestión de su ciclo de vida. La versión disponible para Kubernetes también permite definir todo tipo de recursos sobre infraestructura de Kubernetes (Pods, Deployments, Service, ConfigMap, etc.).

Para entender más sobre Terraform se recomienda una lectura rápida de la sección *"Getting Started with Terraform"* de la referencia [1] *"Terraform Cookbook"*, en la que se introducen los conceptos básicos y terminología de los servicios de orquestación

de Terraform, así como contenido de la referencia [2], *"Terraform: Up and Running"* que describe las capacidades básicas del lenguaje HCL de Terraform.

2 - Terraform con OpenStack

En la Figura 1 se representa el escenario inicial que se usará en esta práctica, donde se incorpora un nodo más a la infraestructura de OpenStack que hará de proveedor de recursos con Terraform.

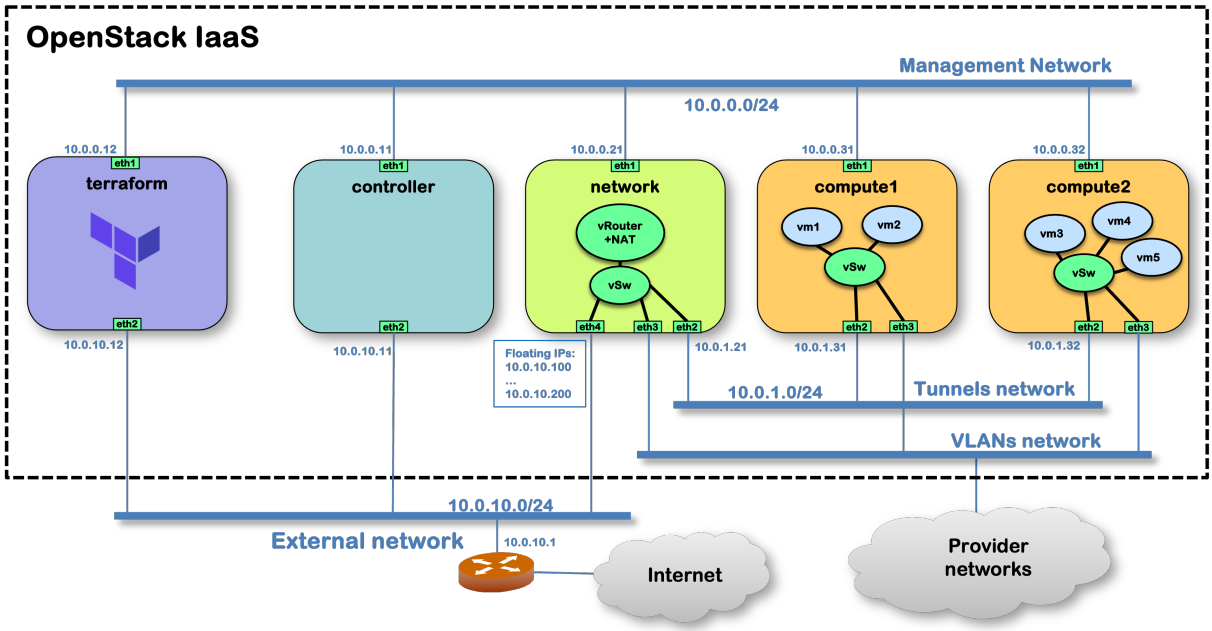


Figura 1: Escenario de la práctica de OpenStack con Terraform

Para comenzar, arranque el escenario virtual de OpenStack de la forma habitual:

- Acceda a un ordenador del laboratorio, entre en su cuenta, abra un terminal y ejecute:

```
/lab/cnvr/bin/get-openstack-tutorial.sh
cd /mnt/tmp/openstack_lab-antelope_4n_classic_ovs-v04
sudo vnx -f openstack_lab.xml --create
sudo vnx -f openstack_lab.xml -x start-all,load-img
sudo vnx_config_nat ExtNet $(ip route | grep default | cut -d" " -f 5)
```

- Arranque una ventana de navegador y acceda al *Dashboard*:

```
http://10.0.10.11
```

- Por último, arranque el siguiente escenario de VNX que desplegará un nuevo nodo *terraform* y lo conectará a la infraestructura de OpenStack para hacer de proveedor de servicios con Terraform:

```
sudo vnx -f openstack_lab-terraform.xml --create
sudo vnx -f openstack_lab-terraform.xml -x install-terraform
```

- Una vez arranque la nueva máquina denominada *terraform*, prueba que se instaló convenientemente el cliente de Terraform con el siguiente comando:

```
terraform --version
```

- Si no se instaló correctamente, introduzca de nuevo el comando para instalar Terraform en la máquina *terraform*:

```
sudo vnx -f openstack_lab-terraform.xml -x install-terraform
```

- Destacar que existe una carpeta compartida entre el *host* (directorio */mnt/tmp/openstack_lab-antelope_4n_classic_ovs-v04/shared*) y la máquina *terraform* (directorio */shared*) que será especialmente útil para enlazar los

ficheros de especificación de recursos de Terraform que sean definidos desde el *host* para poderlos ejecutar luego desde la máquina *terraform.* Se recomienda abrir desde el *host* el directorio de trabajo */mnt/tmp/openstack_lab-antelope_4n_classic_ovs-v04/shared* desde el IDE de Visual Studio Code (VS Code) para editar las especificaciones de Terraform. Se recomienda además el uso de la extensión *"HashiCorp Terraform"* para ayudar a detectar problemas de sintaxis en la especificación de ficheros en Terraform.

- **Nota 2.1:** recuerda que está trabajando sobre un directorio temporal por lo que no olvide copiar esos ficheros a un directorio local no volátil.
- **Nota 2.2:** en el espacio de Moodle de la asignatura encontrará un archivo comprimido denominado "terraform_files_p1.4.tar.gz" con recursos que se utilizarán en los diferentes apartados de la práctica. Encontrará tres directorios ("apartado2-1", "apartado2-2" y "apartado2-3") con los recursos necesarios para cada apartado. Se recomienda que según se vaya avanzando secuencialmente por cada apartado del enunciado de la práctica se vayan incluyendo en la carpeta */mnt/tmp/openstack_lab-antelope_4n_classic_ovs-v04/shared* los ficheros de cada directorio asociado.

La documentación oficial de Terraform para OpenStack puede encontrarse en [3].

2.1 - Despliegue de máquinas virtuales mediante Terraform

Arranque el escenario de demostración mediante:

```
sudo vnx -f openstack_lab.xml -x create-demo-scenario
```

Defina el siguiente fichero inicial de especificación, denominándolo *openstack-provider.tf,* donde se detallará la versión del proveedor de OpenStack de Terraform y se incluirán las credenciales de administración sobre OpenStack (similar al script *bin/admin-openrc.sh* que se ejecuta para adquirir las credenciales de administración y operar con el cliente CLI de OpenStack):

```
# Define required providers
terraform {
  required_version = ">= 0.14.0"
  required_providers {
    openstack = {
      source  = "terraform-provider-openstack/openstack"
      version = "~> 1.53.0"
    }
  }
}

# Configure the OpenStack Provider
provider "openstack" {
  user_name   = "admin"
  tenant_name = "admin"
  password    = "xxxx"
  user_domain_name = "Default"
  project_domain_name = "Default"
  auth_url    = "http://controller:5000/v3"
  region      = "RegionOne"
}
```

Nota 2.3: estas especificaciones siguen un formato *.tf* y se recomienda pasar la plantilla antes por un editor de texto de VS Code para evitar problemas de indentación.

Nota 2.4: como facilidad, puede encontrar una copia de este fichero *openstack-provider.tf* en el directorio "apartado2-1" que contiene el archivo comprimido proporcionado en el Moodle.

Para validar la configuración realizada, ejecute el comando "*terraform init"* desde el directorio de trabajo */shared* en la máquina *terraform,* desde donde se desplegarán todos los recursos posteriormente. Además de establecer Terraform como proveedor de servicios de OpenStack, este comando inicial es utilizado para inicializar el entorno de trabajo de Terraform.

Defina ahora la siguiente especificación en un nuevo fichero denominado *vm2.tf* (**Nota 2.5:** fichero proporcionado también como recurso en el Moodle dentro del directorio "apartado2-1" del archivo comprimido) para crear una segunda máquina virtual denominada "vm2" que se conecte a la red "net0" creada ya en OpenStack:

```
# Data source to get the existing images information
data "openstack_images_image_v2" "jammy" {
  name = " jammy-server-cloudimg-amd64-vnx"
}

# Data source to get the existing flavours information
data "openstack_compute_flavor_v2" "m1-smaller" {
  name = "m1.smaller"
}

# Data source to get the existing network information
data "openstack_networking_network_v2" "net0" {
  name = "net0"
}

# Resource to create a new VM
resource "openstack_compute_instance_v2" "vm2" {
  name       = "vm2"
  image_name = data.openstack_images_image_v2.jammy.name
  flavor_name = data.openstack_compute_flavor_v2.m1-smaller.name
  network {
    name = data.openstack_networking_network_v2.net0.name
  }
  user_data = <<-EOT
    #cloud-config
    package_update: true
    packages:
      - apache2
    runcmd:
      - service start apache2
      - echo "VM lauched by Terraform" > /root/info.txt
  EOT
}
```

En primer lugar, se definen bloques de tipo "data" para consultar o recuperar información de recursos ya existentes en OpenStack relativos a imágenes, *flavours* y red a partir del nombre como filtro de coincidencia. Posteriormente se define el bloque de tipo "resource" para poder crear el recurso asociado a una nueva máquina virtual. En el recurso se definen diferentes atributos, algunos introducidos directamente y otros llamando a los datos recuperado previamente. Para tener control sobre los nombres de recursos válidos a consultar o crear con Terraform, consulte la guía oficial [3].

Una vez definida la nueva especificación, puede verificar que la sintaxis definida en el fichero es correcta mediante el siguiente comando:

```
terraform validate
```

Este comando verifica la validez de la configuración para detectar errores sintácticos y de lógica básica en la configuración, pero no interactúa con el proveedor de servicios OpenStack.

Para generar y mostrar un resumen plan de ejecución que detalla los cambios que Terraform realizará si aplicas la configuración ejecute el siguiente comando:

```
terraform plan
```

Este comando muestra cómo afectará en OpenStack el código especificado, detallando los cambios sobre recursos que se crearán, actualizarán o destruirán. También verifica la sintaxis de la configuración realizada. En definitiva, muestra una vista previa de los cambios que se van a aplicar en OpenStack.

Una vez verificada la sintaxis y plan de ejecución del código, puedes ejecutar el siguiente comando para aplicar los cambios:

```
terraform apply
```

Comprueba desde el Dashboard de OpenStack que se ha creado la máquina virtual correctamente. Asimismo, puede obtener el listado de los recursos consultados y definidos con Terraform mediante el siguiente comando:

```
terraform state list
```

Además, puedes consultar la información de un recurso específico desplegado, mediante el siguiente comando (por ejemplo, para la máquina virtual definida):

```
terraform state show openstack_compute_instance_v2.vm2
```

Para borrar el recurso definido y creado desde Terraform, puedes ejecutar el siguiente comando:

```
terraform destroy -target=openstack_compute_instance_v2.vm2
```

Solicitará confirmación antes de borrar el recurso. El argumento “-target” le permite determinar el recurso a borrar. Si no lo utiliza, el comando tratará de borrar todos los recursos creados previamente con Terraform.

Ese argumento “-target” también se puede utilizar para seleccionar recursos concretos en los comandos anteriores *terraform plan* y *terraform apply* para poder evaluar y aplicar nuevos cambios de un único recurso especificado en el fichero de configuración. Vuelva a ejecutar el comando *terraform apply* con la opción “-target” para ver que se crea la máquina virtual correctamente. Finalmente, modifique la plantilla para que a la máquina virtual se le cargue inicialmente la clave pública de una pareja de claves creada previamente de forma manual. Para ello siga las siguientes pautas:

- Desde el directorio de trabajo del *host* (*/mnt/tmp/openstack_lab-antelope_4n_classic_ovs-v04/*), cree un subdirectorio “*keys”* donde se almacenará la clave privada.
- Genere el *keypair* con el CLI de OpenStack redirigiendo la salida al directorio previamente creado con el comando siguiente:

  ```
  openstack keypair create vm2 > keys/vm2
  ```

- Defina en el fichero de especificación *vm2.tf* un nuevo bloque “data” de tipo "openstack_compute_keypair_v2" que recoja el recurso *keypair* previamente creado y actualice el bloque “resource” de la máquina virtual *vm2* para incluir el atributo que haga uso de ese *keypair.*

Para validar y aplicar la actualización de la configuración, vuelva a hacer uso de los comandos *terraform validate, terraform plan* y *terraform apply.*

- **Nota 2.6:** no hace falta que destruya el recurso de la máquina virtual previamente, ya que Terraform se encargará de gestionar su ciclo de vida convenientemente para destruir y recrear la máquina virtual con la nueva información.

Por último, veamos cómo especificar recursos en Terraform para crear y asociar IP flotantes. Defina en el fichero de especificación *vm2.tf* un nuevo bloque "resource" para definir la creación de una nueva IP flotante de la siguiente forma:

```
resource "openstack_networking_floatingip_v2" "fip_vm2" {
  pool = "ExtNet"
}
```

A continuación, tendremos que definir en el fichero de especificación un nuevo recurso de tipo "openstack_compute_floatingip_associate_v2" para asociar la IP flotante a la máquina virtual *vm2* previamente creada. Para ello, incluya lo siguiente, cambiando *<resource_name_x>* y *<resource_name_y>* por los nombres de recursos apropiados:

```
resource "openstack_compute_floatingip_associate_v2" "fip_assoc_vm2" {
  floating_ip = openstack_networking_floatingip_v2.<resource_name_x>.address
  instance_id = openstack_compute_instance_v2.<resource_name_y>.id
}
```

En la especificación anterior, se puede observar cómo se pueden especificar atributos que consulten información de otros recursos especificados, como es el caso del atributo "instance_id" que obtiene el identificador del recurso *<resource_name_y>* previamente definido. De esta forma, se ve una relación de dependencia entre recursos.

Una vez especificados los recursos asociados a la creación y asignación de IP flotante para la máquina virtual *vm2,* aplique la nueva configuración con Terraform y verifique que el proceso es exitoso. Si se asigna correctamente la IP flotante a *vm2,* compruebe finalmente que puede acceder a ella por SSH usando la clave privada creada a partir del *keypair* generado.

2.2 - Definición de variables y paso de parámetros de entrada y salida

Una de las funcionalidades interesantes que proporciona el lenguaje HCL de Terraform es la definición de variables y el paso de parámetros de entrada durante la definición de los recursos, lo que permite crear plantillas genéricas que son particularizadas mediante parámetros proporcionados durante la creación. Asimismo, Terraform proporciona la posibilidad de obtener información sobre recursos creados durante el despliegue, por ejemplo, nombres o las direcciones IP asignadas a cada máquina virtual.

Para definir variables, en las plantillas de configuración de Terraform tienes que especificar bloques de tipo "variable". A continuación, se muestra un ejemplo de cómo definir variables para recoger los nombres de una nueva red y su subred que se desean crear en OpenStack. Guarde esta configuración en un nuevo fichero denominado *variables.tf* (**Nota 2.7:** fichero disponible como recurso en el Moodle dentro del directorio "apartado2-2" del archivo comprimido).

```
variable "nombre_red"{
    description = "Nombre de la red"
    type = string
}

variable "nombre_subred"{
    description = "Nombre de la subred"
```

```
    type = string
}
```

A continuación, se va a crear un nuevo fichero de especificación *network1.tf* para definir los recursos concretos para crear esas nuevas red y subred en OpenStack (**Nota 2.8:** fichero disponible como recurso en el Moodle dentro del directorio "apartado2-2" del archivo comprimido):

```
# network 1
resource "openstack_networking_network_v2" "net1" {
  name = var.nombre_red
}

# subnetwork 1
resource "openstack_networking_subnet_v2" "subnet1" {
  name            = var.nombre_subred
  network_id      = openstack_networking_network_v2.net1.id
  cidr            = "10.1.2.0/24"
  ip_version      = 4
  dns_nameservers = ["8.8.8.8"]
  gateway_ip      = "10.1.2.1"
  allocation_pool {
    start = "10.1.2.2"
    end   = "10.1.2.100"
  }
}
```

En esta última plantilla, podrá observar cómo se llaman a las variables previamente definidas. Además, se puede observar cómo en el atributo "network_id" del recurso "subnet1" se obtiene el identificador de la red referenciando al recurso "net1" previamente definido. Esto último ejemplifica una relación de dependencia entre los dos recursos, de tal manera que el recurso "subnet1" necesita que previamente sea creado el recurso "net1" para obtener su identificador asociado.

Luego, para parametrizar el valor de las variables a la hora de aplicar un fichero de configuración de Terraform con variables, se puede realizar mediante la opción "-var" del comando *terraform apply*. Por ejemplo:

```
terraform apply -var="nombre_red=net1" -var="nombre_subred=subnet1"
```

Una opción alternativa en el comando anterior a pasar como argumentos los parámetros de entrada necesarios por la plantilla de configuración mediante pares variable/valor, es definir en un fichero todos los valores de variables por defecto que se desean utilizar en los despliegues de recursos con Terraform. Un ejemplo conceptual de la definición de este fichero de parámetros con formato *.tfvars* sería el siguiente (**Nota 2.9:** fichero disponible como recurso en el Moodle con nombre *tf-parameters.tfvars* dentro del directorio "apartado2-2" del archivo comprimido):

```
nombre_red = "net1"
nombre_subred = "subnet1"
```

En este caso, el comando de *terraform apply* debe usar la opción "-var-file <fichero-parametros.tfvars>" en lugar de la opción "-var". Por ejemplo:

```
terraform apply -var-file=<fichero-parametros.tfvars>
```

Opcionalmente, si denominas al fichero de parámetros como *terraform.tfvars,* Terraform lo cargará automáticamente cuando ejecutes los comandos *terraform plan* o *terraform apply* sin que tengas que especificar manualmente la opción "-var-file".

Asimismo, las salidas que puede proporcionar la creación de recursos definidos con Terraform se pueden declarar en bloques de tipo "output" y se utiliza la opción "value" para declarar el valor a devolver. Por ejemplo, para obtener el nombre de la red "net1" previamente creada:

```
output "network1_id"{
  value = openstack_networking_network_v2.net1.id
}
```

Se pueden definir estos bloques outputs en el mismo fichero donde defines los recursos (por ejemplo, en *network1.tf*) o en otro fichero aparte dedicado sólo a parámetros de salida (por ejemplo, en *outputs.tf*).

Los valores asignados a las salidas serán devueltos después del comando *terraform apply.* También pueden ser consultados posteriormente con el comando *terraform output*:

```
terraform output
terraform output network_id
```

Modifique el fichero de variables de entrada para que pueda definir el inicio y fin del *pool* de direcciones que se admite en la subred "subnet1". Incorpore en el fichero de parámetros con formato *.tfvars* de los pares variable/valor a utilizar (se recomienda el fichero estándar *terraform.tfvars*). Además, defina bloques "output": el indicado anteriormente para devolver el identificador de la red "net1" y otros dos adicionales para devolver el inicio y fin del *pool* de direcciones definido.

Para verificar esta solución alternativa, borre inicialmente los recursos de red "net1" y subred "subnet1" con la opción *-target* del comando *terraform destroy* y vuelva a crearlos utilizando las nuevas variables, el nuevo fichero de parámetros, y las salidas especificadas ahora en el fichero. Experimente sobre cómo consultar los valores de salida creados con las opciones del comando *terraform output* para obtenerlos en formato JSON.

2.3 - Uso de módulos en Terraform (Opcional)

En Terraform existe el concepto de módulo, que consiste en un conjunto de archivos de especificación (recursos, variables y salidas) agrupados que se pueden reutilizar y parametrizar múltiples veces.

Pongamos un ejemplo para definir un módulo que especifique la estructura de creación de máquinas virtuales que van a ser asignadas a la red *net1* creada en el apartado anterior. Defina un directorio *modules* y un subdirectorio *vm_instance.* En el subdirectorio *vm_instance* definiremos un fichero *main.tf* con la estructura de definición de una máquina virtual, un fichero *variables.tf* para poder parametrizar los valores necesarios en la especificación de la máquina virtual y un fichero *outputs.tf* para definir como salida estándar el identificador de la máquina virtual que se vaya a crear. Luego, definiremos un fichero de especificación *vms-net1.tf* que invocará al módulo definido para crear instancias de máquinas virtuales en la red *net1* parametrizando las variables con valores específicos y usando la salida estándar especificada. **Nota 2.10:** todos estos ficheros *.tf* están disponible como recursos en el Moodle dentro del directorio "apartado2-3" del archivo comprimido.

La estructura de archivos será la siguiente:

/root/shared/

…
├── vms-net1.tf
└── modules/
*   └── vm_instance/*
*   ├── main.tf*
*   ├── variables.tf*

└── *outputs.tf*

Defina las variables a usar por el módulo en el fichero *variables.tf*:

```
variable "name" {
    description = "Name of the VM instance"
    type = string
}
variable "flavor" {
    description = "Flavor of the VM instance"
    type = string
}

variable "image" {
    description = "Image of the VM instance"
    type = string
}
variable "key_pair" {
    description = "Key pair for the VM instance"
    type = string
}
variable "security_groups" {
  description = "List of security groups for the VM instance"
  type = list(string)
}
variable "network_name" {
  description = "Name of the network for the VM instance"
  type = string
}
```

Defina en el módulo la especificación general del recurso de tipo "openstack_compute_instance_v2" para la instanciación de las distintas máquinas virtuales en el fichero *main.tf:*

```
terraform {
  required_providers {
    openstack = {
      source  = "terraform-provider-openstack/openstack"
      version = "~> 1.53.0"
    }
  }
}

resource "openstack_compute_instance_v2" "vm" {
  name            = var.name
  flavor_name     = var.flavor
  image_name      = var.image
```

```
  key_pair        = var.key_pair
  security_groups = var.security_groups
  network {
    name = var.network_name
  }
}
```

Al inicio de la definición de los recursos que use un módulo, es necesario indicar el proveedor de Terraform asociado. En este caso, volverá a usarse el proveedor para OpenStack.

Finalmente, defina el fichero *outputs.tf* para especificar la salida que permita devolver el identificador de la máquina virtual que se vaya a instanciar al usar el módulo:

```
output "instance_id" {
  value = openstack_compute_instance_v2.vm.id
}
```

Finalmente, fuera de la carpeta de módulos, defina el fichero *vms-net1.tf* que invocará al módulo creado para poder crear las máquinas virtuales en la red *net1:*

```
module "vm3" {
  source          = "./modules/vm_instance"
  name            = "vm3"
  flavor          = "m1.tiny"
  image           = "cirros-0.3.4-x86_64-vnx"
  key_pair        = "vm3"
  security_groups = ["default"]
  network_name    = "net1"
}

module "vm4" {
  source          = "./modules/vm_instance"
  name            = "vm4"
  flavor          = "m1.tiny"
  image           = "cirros-0.3.4-x86_64-vnx"
  key_pair        = "vm4"
  security_groups = ["default"]
  network_name    = "net1"
}

output "vm3_id" {
  value = module.vm3.instance_id
}

output "vm4_id" {
  value = module.vm4.instance_id
```

```
}
```

Como podemos ver, se definen bloques de tipo “module” que llaman al módulo “vm_instance” creado y que parametriza los valores asociados a las variables que define el módulo para poder crear las dos instancias de máquinas virtuales. Además, se definen bloques “outputs” que llaman a la salida definida en el módulo para devolver el valor del identificador de las instancias.

Una vez definidos el módulo y el archivo de especificación que lo va a utilizar, para aplicar la configuración, primero hay que reinicializar el entorno de trabajo de Terraform para que sepa reconocer el módulo definido que va a usar también el proveedor de Terraform para OpenStack. Para ello, utilice el siguiente comando:

Reinicialice el entorno de trabajo de Terraform para que reconozca el nuevo módulo definido con el siguiente comando:

```
terraform init -upgrade
```

Si da error, elimine los ficheros de *lock* y metadatos asociados a la inicialización previa de Terraform mediante los siguientes comandos con los siguientes comandos, y después vuelva a introducir el comando anterior para reinicializar el entorno de trabajo de Terraform:

```
rm -rf .terraform
rm -f .terraform.lock.hcl
terraform init -upgrade
```

Finalmente, utilice los comandos *terraform validate, terraform plan* y *terraform apply* para validar y aplicar la nueva configuración asociada al módulo definido y verifique que se crean correctamente las máquinas virtuales según lo especificado.

2.4 - Creación de otros tipos de recursos en OpenStack (Opcional)

Terraform permite crear todo tipo de recursos de OpenStack, no solamente máquinas virtuales y redes. En [3] puede encontrar toda la lista de tipos de recursos que se pueden crear en OpenStack mediante Terraform. Este diseño modular no sólo permite reutilizar código de especificación de recursos, sino que además facilita la organización en la definición de los distintos recursos que se vayan a configurar.

En este apartado se experimentará con la creación de otros tipos de recursos utilizando Terraform. En particular, se tratará de reproducir el escenario de red planteado en la práctica 2.1 mediante plantillas de configuración en Terraform. Para ello, se recomienda seguir los pasos siguientes:

- Script para crear parejas de claves (*keypairs*) mediante uso del comando “openstack keypair create”.
- Crear una plantilla en Terraform (*networks.tf*) para crear las redes internas net1 y net2 (y sus subredes) utilizando los mismos parámetros utilizados para definir esas redes mediante los comandos del cliente nativo de OpenStack. Para ello, trate de definir un módulo que permita estructurar de forma general la definición de los recursos y variables necesarios para crear una red y subred, para que luego el módulo pueda ser reutilizado desde el fichero *networks.tf* para parametrizar fácilmente los valores de las redes y subredes a configurar.
- Crear una plantilla en Terraform (*vms.tf*) para crear las diferentes máquinas virtuales y asignarlas a sus redes concretas. Tendrán que hacer uso de la imagen *cirros* y el *flavour m1.tiny,* así como de los *keypairs* previamente creados. Defina y asocie con Terraform direcciones IP flotantes para las máquinas. Puede probar a definir un módulo para definir la estructura general del recurso asociado

a una máquina virtual que sea aprovechable para la configuración de las distintas máquinas virtuales.

- Defina una plantilla Terraform (*r0.tf*) para crear un router que interconecte la red ExtNet y las redes net1 y net2.

Consulte la documentación para conocer la sintaxis de los comandos necesarios. La referencia completa sobre el lenguaje HCL de Terraform para OpenStack puede encontrarse en [3]. Adicionalmente, en la carpeta de ejemplos del repositorio GitHub oficial *"Terraform OpenStack Provider"* [4] puede encontrar algunos ejemplos de interés.

2.5 – Exploración de otras funcionalidades de Terraform para OpenStack (Información Extra)

Además de las funcionalidades básicas de Terraform para desplegar y gestionar recursos en OpenStack, es interesante explorar otras capacidades que ofrece el lenguaje HCL y del proveedor de Terraform para OpenStack para automatizar y flexibilizar la configuración de infraestructuras en la nube. Por ejemplo:

- El uso de *cloud-init* para la autoconfiguración de las máquinas virtuales.
- Las capacidades del lenguaje HCL para especificar escenarios con un número variable de máquinas virtuales definido por el usuario en tiempo de creación.
- Explotar las capacidades del lenguaje HCL para la definición modular de los recursos.
- Gestión de referencias y dependencias entre recursos. Como ya hemos visto, Terraform permite usar referencias entre recursos para que, por ejemplo, una IP flotante se asocie a una instancia de máquina virtual disponible o para que una subred encuentre el identificador de la red dependiente. Al establecer referencias entre recursos, Terraform detecta automáticamente la dependencia entre ellos y sabe aprovisionarlos en el orden adecuado. Existe también la posibilidad de establecer dependencias explícitas cuando no se incluyen referencias directas entre recursos mediante el uso de sentencias "depends_on" en los bloques "resources" de definición de recursos con Terraform.
- Analizar la posibilidad de creación dinámica de otro tipo de recursos desde Terraform como *keypairs* o *security groups.*

3 - Terraform con Kubernetes

Dado que Terraform se va a utilizar como herramienta base para el despliegue del de recursos de Kubernetes para el escenario del trabajo final de la asignatura, es interesante explorar más a fondo funcionalidades que se utilizarán en dicho trabajo.

En la Figura 2 se representa el escenario final que se usará en esta práctica y en el trabajo final, donde se incorpora un nodo más a la infraestructura de Kubernetes que hará de proveedor de recursos con Terraform. En este caso, Kubernetes utiliza el módulo de red o CNI (*Container Network Interface)* Calico [8], necesario para utilizar recursos de tipo *NetworkPolicy* utilizados en el apartado 3.3.

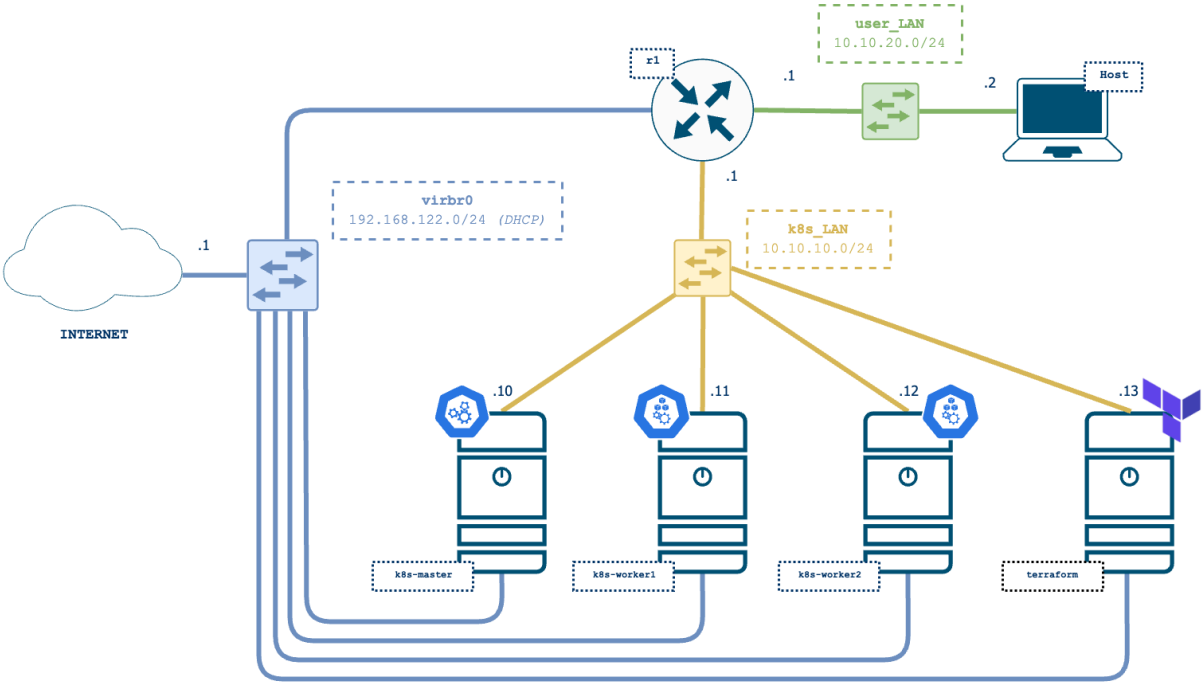


Figura 2: Escenario de la práctica de Kubernetes con Terraform

Para comenzar, arranque el escenario virtual de Kubernetes de la forma habitual:

- Acceda a un ordenador del laboratorio, entre en su cuenta, abra un terminal y ejecute:

```
/lab/cnvr/bin/get-k8s-calico
cd /mnt/tmp/k8s_lab-calico-v02
./bin/start-k8s-calico
```

- Por último, arranque el siguiente escenario de VNX que desplegará un nuevo nodo *terraform* y lo conectará a la infraestructura de Kubernetes para hacer de proveedor de servicios con Terraform:

```
sudo vnx -f k8s_lab-terraform.xml --create
sudo vnx -f k8s_lab-terraform.xml -x install-terraform
```

- Una vez arranque la nueva máquina denominada *terraform*, prueba que se instaló convenientemente el cliente de Terraform con el siguiente comando:

```
terraform --version
```

- Si no se instaló correctamente, introduzca de nuevo el comando para instalar Terraform en la máquina *terraform*:

```
sudo vnx -f k8s_lab-terraform.xml -x install-terraform
```

- Por último, para cargar las credenciales de administración de clúster de Kubernetes en el hosts y exportarlas a la máquina *terraform*, ejecute los siguientes comandos:

```
./bin/get-master-kubeconf
./bin/set-terraform-kubeconf
```

- Destacar que existe una carpeta compartida entre el *host* (directorio */mnt/tmp/k8s_lab-calico-v02/shared*) y la máquina *terraform* (directorio */shared*) que será especialmente útil para enlazar los ficheros de especificación de recursos de Terraform que sean definidos desde el *host* para poderlos ejecutar luego desde la máquina *terraform.* Se recomienda abrir desde el *host* el directorio de trabajo */mnt/tmp/k8s_lab-calico-v02/shared* desde el IDE de Visual Studio Code (VS Code) para editar las especificaciones de Terraform. Se recomienda

además el uso de la extensión *"HashiCorp Terraform"* para ayudar a detectar problemas de sintaxis en la especificación de ficheros en Terraform.

- **Nota 3.1:** recuerda que está trabajando sobre un directorio temporal por lo que no olvide copiar esos ficheros a un directorio local no volátil.
- **Nota 3.2:** en el espacio de Moodle de la asignatura encontrará un archivo comprimido denominado "terraform_files_p1.4.tar.gz" con recursos que se utilizarán en los diferentes apartados de la práctica. Encontrará X directorios ("apartado3-1", "apartado3-2" y "apartado3-3") con los recursos necesarios para cada apartado. Se recomienda que según se vaya avanzando secuencialmente por cada apartado del enunciado de la práctica se vayan incluyendo en la carpeta */mnt/tmp/k8s_lab-calico-v02/shared* los ficheros de cada directorio asociado.

La documentación oficial de Terraform para Kubernetes puede encontrarse en [7].

3.1 - Creación de recursos en Kubernetes mediante Terraform

Defina el siguiente fichero inicial de especificación, denominándolo *k8s-provider.tf,* donde se detallará la versión del proveedor de Kubernetes de Terraform y se incluirán las credenciales de administración sobre Kubernetes para poder operar el clúster de Kubernetes desde Terraform:

```
# Define required providers
terraform {
  required_providers {
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "2.23.0"
    }
  }
}

# Configure the OpenStack Provider
provider "kubernetes" {
  config_path = "~/.kube/config"
}
```

Nota 3.3: estas especificaciones siguen un formato *.tf* y se recomienda pasar la plantilla antes por un editor de texto de VS Code para evitar problemas de indentación.

Nota 3.4: como facilidad, puede encontrar una copia de este fichero *k8s-provider.tf* en el archivo comprimido proporcionado en el Moodle.

Para validar la configuración realizada, ejecute el comando "*terraform init"* desde el directorio de trabajo */shared* en la máquina *terraform,* desde donde se desplegarán todos los recursos posteriormente. Además de establecer Terraform como proveedor de servicios de Kubernetes, este comando inicial es utilizado para inicializar el entorno de trabajo de Terraform.

Defina ahora la siguiente especificación en un nuevo fichero denominado *deployment.tf* (**Nota 3.5:** fichero proporcionado también como recurso en el Moodle dentro del directorio "apartado3-1" del archivo comprimido) para crear un recurso *Deployment* en Kubernetes que, de forma equivalente a lo visto en la práctica 1.3, desplegará tres réplicas de Pods de servidores web Nginx.

```
resource "kubernetes_deployment_v1" "web_servers" {
  metadata {
    name = "nginx-deployment"
    labels = {
```

```
      app = "nginx"
    }
  }

  spec {
    replicas = 3
    selector {
      match_labels = {
        app = "nginx"
      }
    }
    template {
      metadata {
        labels = {
          app = "nginx"
        }
      }
      spec {
        container {
          name  = "nginx-server"
          image = "nginx"

          port {
            container_port = 80
          }
        }
      }
    }
  }
}
```

Una vez definida la nueva especificación, tal como hizo en el escenario con OpenStack, verifique que la sintaxis definida en el fichero es correcta mediante el siguiente comando:

```
terraform validate
```

Para generar y mostrar un resumen plan de ejecución de cambios que Terraform realizará si se aplica la configuración, ejecute el siguiente comando:

```
terraform plan
```

Una vez verificada la sintaxis y plan de ejecución del código, aplique los cambios con el siguiente comando:

```
terraform apply
```

Comprueba desde el cliente “kubectl” de Kubernetes que se ha creado correctamente el recurso *Deployment* y que existen 3 réplicas de *Pods*. Asimismo, puede obtener el listado de los recursos consultados y definidos con Terraform mediante el siguiente comando:

```
terraform state list
```

Además, puedes consultar la información del recurso específico desplegado, mediante el siguiente comando:

```
terraform state show kubernetes_deployment_v1.web_servers
```

Para borrar el recurso definido y creado desde Terraform, puedes ejecutar el siguiente comando:

```
terraform destroy -target=kubernetes_deployment_v1.web_servers
```

Solicitará confirmación antes de borrar el recurso. Como ya se ha visto, el argumento “-target” permite determinar el recurso a borrar. Si no se incluye, el comando tratará de borrar todos los recursos creados previamente con Terraform.

Como ya se indicó previamente, ese argumento “-target” también se puede utilizar para seleccionar recursos concretos en los comandos anteriores *terraform plan* y *terraform apply* para poder evaluar y aplicar nuevos cambios de un único recurso especificado en la plantilla de especificación. Vuelva a ejecutar el comando *terraform apply* con la opción “-target” para ver que se crea el recurso *Deployment* correctamente. Finalmente, modifique la plantilla para que el *Deployment* tenga ahora cuatro réplicas. Para validar y aplicar la actualización de la configuración, vuelva a hacer uso del comando *terraform apply* sin necesidad de destruir de nuevo el recurso *Deployment*, ya que Terraform y Kubernetes se encargarán de ajustar dinámicamente el número de réplicas deseado.

A continuación, veamos cómo hacer uso de bloques de tipo “variable” para definir variables de entrada sobre recursos definidos para Kubernetes. A continuación, se muestra un ejemplo de cómo definir una variable para recoger el número de réplicas de *Pods* para el recurso *Deployment*. Guarde esta configuración en un nuevo fichero denominado *variables.tf* (**Nota 3.6:** fichero disponible como recurso en el Moodle dentro del directorio “apartado3-1” del archivo comprimido).

```
variable "num_replicas"{
    description = "Número de réplicas"
    type = number
    default = 3
}
```

A continuación, modifique el fichero de especificación *deployment.tf* para que el número de replicas provenga del nuevo bloque de tipo “variable”, utilizando la expresión “var.num_replicas” en el campo *replicas*.

Luego, para parametrizar el valor de las variables a la hora de aplicar un fichero de configuración de Terraform con variables, podemos utilizar la opción “-var” del comando *terraform apply*, tal que:

```
terraform apply -var="num_replicas=5"
```

Pruebe a aplicar esa modificación en la especificación del recurso *Deployment* para parametrizar el número de réplicas. Como ya se detalló en la sección de la práctica dedicada a OpenStack, una opción alternativa a parametrizar los valores de las variables de entrada es definir en un fichero todos los valores de variables que se desean utilizar. Un ejemplo conceptual de la definición de este fichero de parámetros con formato *.tfvars* sería el siguiente:

```
num_replicas = 5
```

Como ya se vio, en este caso, el comando de *terraform apply* debe usar la opción “-var-file <fichero-parametros.tfvars>”. Por ejemplo:

```
terraform apply -var-file=<fichero-parametros.tfvars>
```

Opcionalmente, si se denomina al fichero de parámetros como *terraform.tfvars,* Terraform lo cargará automáticamente al ejecutar el comando *terraform apply* sin que tener que especificar manualmente la opción “-var-file”.

Finalmente, probemos con obtener valores de salidas que puede proporcionar la creación de recursos de Kubernetes definidos con Terraform, declarando bloques de tipo “output” con opción “value” para declarar el valor a devolver. Por ejemplo, para obtener el nombre del recurso *Deployment* y el número de réplicas deseado defina los siguientes bloques “output”:

```
output "deployment_name" {
  value = kubernetes_deployment_v1.web_servers.metadata[0].name
}

output "deployment_replicas" {
```

```
  value = kubernetes_deployment_v1.web_servers.spec[0].replicas
}
```

Pruebe a definir estos bloques “output” en el mismo fichero *deployment.tf* o en otro fichero aparte dedicado sólo a parámetros de salida (por ejemplo, en *outputs.tf*).

Los valores asignados a las salidas serán devueltos después del comando *terraform apply.* También pueden ser consultados posteriormente con el comando *terraform output*:

```
terraform output
terraform output deployment_name
terraform output deployment_replicas
```

Veamos cómo podría usarse un bloque “output” para obtener también el nombre de los *Pods* creados al especificar el recurso *Deployment.* Para ello, sería necesario definir primero un bloque “data” que permita previamente consultar la lista de *Pods* asociada al recurso *Deployment,* utilizando como filtro de búsqueda la etiqueta “app=nginx” definida para los *Pods.* Luego, ya se puede definir el bloque “output” que haga uso del bloque “data” anterior para poder iterar el listado de *Pods* y obtener sus nombres. A continuación, dispone de la especificación asociada a la configuración previamente indicada para definirla en un nuevo fichero denominado *output_pod_names.tf*:

```
data "kubernetes_resources" "nginx_pods" {
  api_version = "v1"
  kind        = "Pod"
  label_selector = "app=nginx"
  depends_on = [
    kubernetes_deployment_v1.web_servers
  ]
}

output "nginx_pod_names" {
  value = [
    for pod in data.kubernetes_resources.nginx_pods.objects : pod.metadata.name
  ]
}
```

Una vez aplicados los bloques “data” y “output” anteriores (ejecute comando “terraform apply”), verifique que puede obtener los nombres de los *Pods.*

3.2 - Referencias entre recursos de Kubernetes en Terraform

En el Service usar bloques data para referenciar a Deployment en lugar de labels de Kubernetes.

A continuación, vamos a crear un recurso *Service* de tipo NodePort mediante Terraform que permita acceder a los servidores web desde fuera del clúster de Kubernetes. Veremos que hay diferentes utilidades y formas de referenciar recursos de Kuberentes en Terrafrom.

Siguiendo la forma habitual de referenciar recursos en Kubernetes con *labels* o etiquetas, define la siguiente especificación en un nuevo fichero denominado *service-with-label.tf* (**Nota 3.7:** fichero proporcionado también como recurso en el Moodle dentro del directorio “apartado3-2” del archivo comprimido) y aplíquela con el comando “terraform apply -target=kubernetes_service_v1.web_service_with_label”:

```
resource "kubernetes_service_v1" "web_service_with_label" {
  metadata {
    name = "nginx-service"
  }
  spec {
```

```
    selector = {
      app = "nginx"
    }
    port {
      port        = 80
      target_port = 80
      node_port   = 30001
    }
    type = "NodePort"
  }
}
```

Como vemos, en la especificación del recurso *Service*, en el selector indicamos la etiqueta para identificar los *Pods* asociados. Una vez aplicado, verifique con el cliente "kubectl" que se ha creado correctamente el recurso *Service* y que puede realizar peticiones a los servidores web desde fuera del clúster de Kubernetes. Si estas familiarizado con las referencias mediante etiquetas en Kubernetes esta alternativa es bastante cómoda. El problema está en que hay que tener cuidado con mantener los valores de etiquetas sincronizados y en caso de cambios en etiquetas es un mecanismo propenso a errores.

Como alternativa, veremos que Terraform permite realizar referencias directas al recurso objetivo. Para ello, hay que navegar por los diferentes atributos de la especificación del recurso objetivo ya definido hasta llegar al atributo *labels* y especificar el acceso al mismo en una expresión de una línea en el recurso desde donde queremos hacer la referencia, pero sin necesidad de declarar las claves y valores de las propias *labels*. Veamos un ejemplo. Borre primero el recurso *Service* declarado anteriormente con el comando "terraform destroy -target=kubernetes_service_v1.web_service_with_label". A continuación, define la siguiente especificación en un nuevo fichero denominado *service-with-reference.tf* (**Nota 3.8:** fichero proporcionado también como recurso en el Moodle dentro del directorio "apartado3-2" del archivo comprimido) y aplíquela con el comando "terraform apply -target=kubernetes_service_v1.web_service_with_reference":

```
resource "kubernetes_service_v1" "web_service_with_reference" {
  metadata {
    name = "nginx-service"
  }
  spec {
    selector = kubernetes_deployment_v1.web_servers.spec[0].template[0].metadata[0].labels
    port {
      port        = 80
      target_port = 80
      node_port   = 30001
    }
    type = "NodePort"
  }
}
```

Nota 3.9: Los índices [0] permiten acceder al primer bloque de cada tipo de atributo, haya sólo uno o más de uno.

Una vez aplicado, verifique de nuevo con el cliente "kubectl" que se ha creado correctamente el recurso *Service* y que puede realizar peticiones a los servidores web desde fuera del clúster de Kubernetes. Vemos que esta alternativa permite inferir correctamente la dependencia entre recursos, y en caso de cambios en las etiquetas del recurso dependiente, el campo *selector* del recurso *Service* se actualizaría correctamente. El problema es que la expresión del campo *selector* es más larga y menos intuitiva que en la alternativa anterior.

Además, como tercera alternativa, existe la posibilidad de consultar previamente el recurso dependiente con el que queremos establecer la relación mediante la definición de un bloque de tipo "data" en Terraform. Esta sería la opción idónea en

caso de que el recurso dependiente no haya sido creado/gestionado por nosotros desde Terraform y que necesitemos consultar su información previamente, siempre que sepamos cómo identificarlo (mediante campos como nombre de recurso, tipo de recurso y *namespace*) para que el proveedor de Terraform pueda consultarlo. Para ello, veamos otro ejemplo. Borre primero el recurso Service declarado anteriormente con el comando "terraform destroy -target=kubernetes_service_v1.web_service_with_reference". A continuación, define la siguiente especificación en un nuevo fichero denominado *service-with-datasource.tf* (**Nota 3.10:** fichero proporcionado también como recurso en el Moodle dentro del directorio "apartado3-2" del archivo comprimido) y aplíquela con el comando "terraform apply -target=kubernetes_service_v1.web_service_with_datasource":

```
data "kubernetes_resource" "web_servers" {
  api_version = "apps/v1"
  kind        = "Deployment"
  metadata {
    name      = "nginx-deployment"
    namespace = "default"
  }
}

resource "kubernetes_service_v1" "web_service_with_datasource" {
  metadata {
    name = "nginx-service"
  }
  spec {
    selector = data.kubernetes_resource.web_servers.object.spec.template.metadata.labels
    port {
      port        = 80
      target_port = 80
      node_port   = 30001
    }
    type = "NodePort"
  }
}
```

Vea cómo ahora definimos el bloque "data" para obtener la información del recurso *Deployment,* y en el bloque "resource" declaramos el recurso *Service* con el campo *selector* apuntando a las etiquetas que pueda disponer el recurso *Deployment* en la especificación de sus *Pods,* incluyendo una expresión similar a la de la alternativa anterior. Una vez aplicado, verifique otra vez con el cliente "kubectl" que se ha creado correctamente el recurso *Service* y que puede realizar peticiones a los servidores web desde fuera del clúster de Kubernetes.

Por último, una solución que ofrece Terraform es el poder incluir bloques de tipo "locals" para poder definir a nivel global las *labels* u otros campos de interés que puedan ser compartidos por diferentes recursos a especificar en la plantilla. Veámoslo en un nuevo ejemplo. Borre primero los recursos *Deployment* y *Service* declarados anteriormente con el comando "terraform destroy -target=kubernetes_deployment_v1.web_servers -target=kubernetes_service_v1.web_service_with_datasource". A continuación, define la siguiente especificación en un nuevo fichero denominado *deployment-and-service-with-locals.tf* (**Nota 3.11:** fichero proporcionado también como recurso en el Moodle dentro del directorio "apartado3-2" del archivo comprimido) y aplíquela con el comando "terraform apply -target=kubernetes_deployment_v1.web_servers_with_locals -target=kubernetes_service_v1.web_service_with_locals":

```
locals {
  web_labels = {
    app = "nginx"
  }
```

```
}

resource "kubernetes_deployment_v1" "web_servers_with_locals" {
  metadata {
    name = "nginx-deployment"
    labels = local.web_labels
  }

  spec {
    replicas = 3
    selector {
      match_labels = local.web_labels
    }
    template {
      metadata {
        labels = local.web_labels
      }
      spec {
        container {
          name  = "nginx-server"
          image = "nginx"
          port {
            container_port = 80
          }
        }
      }
    }
  }
}

resource "kubernetes_service_v1" "web_service_with_locals" {
  metadata {
    name = "nginx-service"
  }
  spec {
    selector = local.web_labels
    port {
      port        = 80
      target_port = 80
      node_port   = 30001
    }
    type = "NodePort"
  }
}
```

Observe cómo en el nuevo bloque “locals” especificamos las *labels* a utilizar en los recursos *Deployment* y *Service* definidos. Luego, las etiquetas definidas en el bloque “locals” son obtenidas y utilizadas tanto desde el apartado *label* de los campos *metadata* en la especificación del recurso *Deployment* como desde los campos *selector* de los recursos *Deployment* y *Service*, indicando para ello la expresión “local.web_labels”. Una vez aplicados los cambios, compruebe de nuevo con el cliente “kubectl” que se han creado correctamente los recursos y que puede realizar peticiones a los servidores web desde fuera del clúster de Kubernetes.

3.3 - Recursos de tipo *NetworkPolicy* con Terraform

A continuación, se va a analizar un nuevo tipo de recurso de Kubernetes denominado *NetworkPolicy.* Las *NetworkPolicy* son recursos de Kubernetes que permiten definir qué tráfico de red está autorizado hacia y desde los *Pods* del clúster, tanto de entrada (*Ingress*) como de salida (*Egress*). Estas políticas permiten controlar las comunicaciones entre Pods del clúster y, cuando corresponda, el acceso a los Pods desde sistemas externos, así como las comunicaciones que los Pods inician hacia destinos externos. Funcionan mediante selectores que identifican los *Pods* afectados y reglas que especifican los orígenes, destinos, protocolos y puertos permitidos.

Su funcionamiento es similar al de un cortafuegos o *firewall*: una política puede permitir determinadas comunicaciones y restringir las demás. Sin embargo, hay que tener en cuenta que las políticas de Kubernetes son aditivas: cuando varias políticas seleccionan un mismo Pod, el tráfico permitido es la unión de las reglas de autorización de todas ellas.

Para que las *NetworkPolicy* tengan efecto es necesario utilizar un CNI que las implemente. En el escenario de Kubernetes utilizamos Calico para ello.

En este apartado de la práctica se comprobará cómo afectan estas políticas a las comunicaciones entre Pods y al acceso externo a los servidores web desplegados previamente en el apartado anterior, utilizando para ello Terraform.

3.3.1 - Despliegue de Pod de pruebas y comprobación de comunicación inicial con CNI de Calico

En primer lugar, se desplegará un Pod de pruebas (tester) que servirá como origen de las pruebas de conectividad. Para ello, defina la siguiente especificación en un nuevo fichero denominado *pod-tester.tf* (**Nota 3.12:** fichero proporcionado también como recurso en el Moodle dentro del directorio “apartado3-3” del archivo comprimido) y aplíquela con el comando “terraform apply -target=kubernetes_pod_v1.tester”:

```
resource "kubernetes_pod_v1" "tester" {
  metadata {
    name      = "tester"
    namespace = "default"

    labels = {
      app = "tester"
    }
  }

  spec {
    container {
      name    = "tester"
      image   = "dougbtv/centos-network"
      command = ["sleep", "3600"]
    }
  }
}
```

Comprueba con el cliente “kubectl” que el Pod se encuentra en ejecución, Comprueba ahora la conectividad desde el Pod *tester* a los Pods de los servidores web asociados al Deployment, mediante pruebas de tráfico ICMP con la herramienta ping y de peticiones HTTP (al puerto 80 TCP) con la herramienta curl. Utiliza el comando “kubectl get pods -o wide” para obtener las direcciones IP de los Pods para hacer las pruebas.

En este punto no se han aplicado restricciones mediante *NetworkPolicy*. Por tanto, las comunicaciones entre los Pods deberían estar permitidas por la configuración de red del escenario.

Cuando dos Pods están alojados en diferentes nodos worker, el CNI de Calico transporta el tráfico entre ellos mediante túneles IP-in-IP. Esto permite encapsular los paquetes originales para encaminarlos entre los nodos worker. Para observar este comportamiento, puede realizar una captura de tráfico en las interfaces e1 de los nodos worker del escenario de red con Wireshark, generando tráfico entre Pods ubicados en nodos worker distintos mientras se realiza la captura de tráfico. Identifique los paquetes IP-in-IP y observe las direcciones IP de los nodos que actúan como extremos del túnel. Puedes comparar el tráfico encapsulado con el tráfico original intercambiado entre los Pods.

3.3.2 - Denegación de todo el tráfico hacia y desde Pods

A continuación, se aplicará un recurso *NetworkPolicy* que seleccione todos los Pods del *namespace default* y deniegue tanto el tráfico de entrada como el de salida. Defin la siguiente especificación del recurso *NetworkPolicy* en un fichero *network-policy-deny-all.tf* (**Nota 3.13:** fichero proporcionado también como recurso en el Moodle dentro del directorio "apartado3-3" del archivo comprimido) y aplíquela con el comando "terraform apply -target=kubernetes_network_policy_v1.default_deny_all":

```
resource "kubernetes_network_policy_v1" "default_deny_all" {
  metadata {
    name      = "default-deny-all"
    namespace = "default"
  }

  spec {
    pod_selector {}

    policy_types = [
      "Ingress",
      "Egress"
    ]
  }
}
```

La política no contiene reglas de autorización para ninguno de los dos sentidos. Por ello, el tráfico de entrada y salida de los *Pods* seleccionados queda denegado, salvo las comunicaciones que puedan permitirse por otras *NetworkPolicy* aplicables.

Con los siguientes comandos del cliente "kubectl" puede verificar la configuración de *NetworPolicy* aplicada:

```
kubectl get networkpolicy
kubectl describe networkpolicy default-deny-all
```

Vuelva a realizar pruebas de conectividad y compruebe cómo la comunicación deja de funcionar tanto para el tráfico entre Pods como para el tráfico HTTP desde el exterior del clúster de K8s hacia los servidores web utilizando el recurso *Service,* teniendo en cuenta en este último caso cómo una petición HTTP dirigida a un *Service* puede verse afectada por las políticas aplicables a los *Pods* de destino.

No elimines la política *default-deny-all.* En el siguiente apartado se añadirán nuevas *NetworkPolicy* para comprobar cómo funcionan las reglas de autorización cuando existe una política de denegación general.

Recuerda que las *NetworkPolicy* son aditivas: el tráfico permitido por cualquiera de las políticas aplicables queda autorizado, aunque otra política no lo permita expresamente. Por tanto, las nuevas reglas permitirán determinadas comunicaciones sin eliminar las restricciones sobre el resto del tráfico.

3.3.3 - Comprobación del comportamiento *stateful* de *NetworkPolicy*

Los recursos *NetworkPolicy* permiten autorizar el tráfico de entrada y de salida de forma independiente. Sin embargo, el seguimiento del estado de las conexiones hace que las respuestas de una comunicación previamente autorizada puedan circular en sentido inverso sin necesitar una regla explícita adicional (comportamiento *stateful*).

Se comprobará este comportamiento mediante dos políticas: una que permite el tráfico de salida del *Pod* de pruebas hacia los servidores web y otra que permite el tráfico de entrada a los servidores web desde el *Pod* de pruebas.

Para permitir únicamente el tráfico de salida del *Pod* de pruebas hacia los *Pods de* los servidores web, defina la siguiente especificación de recurso *NetworkPolicy* en un fichero *network-policy-egress-tcp-tester.tf* (**Nota 3.14:** fichero proporcionado

también como recurso en el Moodle dentro del directorio “apartado3-3” del archivo comprimido) y aplíquela con el comando “terraform apply -target=kubernetes_network_policy_v1.tester_egress”:

```
resource "kubernetes_network_policy_v1" "tester_egress" {
  metadata {
    name      = "tester-egress"
    namespace = "default"
  }

  spec {
    pod_selector {
      match_labels = {
        app = "tester"
      }
    }

    policy_types = ["Egress"]

    egress {
      to {
        pod_selector {
          match_labels = {
            app = "nginx"
          }
        }
      }

      ports {
        protocol = "TCP"
        port     = "80"
      }
    }
  }
}
```

Esta política permite que el Pod identificado con la etiqueta “app = tester” inicie conexiones TCP hacia el puerto 80 de los Pods identificados con la etiqueta “app = nginx”. No autoriza otras comunicaciones de salida que no estén permitidas por otras políticas.

Para permitir únicamente el tráfico de entrada a los servidores web desde el Pod de pruebas, defina la siguiente especificación de recurso *NetworkPolicy* en un fichero *network-policy-ingress-tcp-web-servers.tf* (**Nota 3.15:** fichero proporcionado también como recurso en el Moodle dentro del directorio “apartado3-3” del archivo comprimido) y aplíquela con el comando “terraform apply -target=kubernetes_network_policy_v1.nginx_ingress”:

```
resource "kubernetes_network_policy_v1" "nginx_ingress" {
  metadata {
    name      = "nginx-ingress"
    namespace = "default"
  }

  spec {
    pod_selector {
      match_labels = {
        app = "nginx"
      }
    }

    policy_types = ["Ingress"]

    ingress {
      from {
        pod_selector {
          match_labels = {
            app = "tester"
          }
```

```
      }
    }

    ports {
      protocol = "TCP"
      port     = "80"
    }
  }
 }
}
```

Esta política permite el tráfico TCP destinado al puerto 80 de los Pods de Nginx cuando el origen es un Pod con la etiqueta “app = tester”.

Una vez configuradas estas reglas, compruebe que el tráfico de consulta HTTP iniciada desde el *Pod tester* hacia *Pods* de los servidores web es exitosa. Pruebe también que, por ejemplo, el tráfico ICMP producto de un ping desde el *Pod tester* hacia *Pods* de los servidores web sigue sin funcionar.

Los resultados permiten comprobar que las *NetworkPoliciy* distinguen entre el tráfico que inicia una conexión y el tráfico de respuesta:

- Regla Egress en el Pod de pruebas: permite iniciar conexiones TCP hacia el puerto 80 de los Pods de Nginx. No es necesaria una regla Ingress adicional en en el Pod de pruebas para permitir recibir las respuestas de esas conexiones, ya que las *NetworkPolicy* implementan un comportamiento *stateful* y permiten el tráfico de retorno de las conexiones previamente autorizadas. No obstante, si se mantiene aplicada la política *default-deny-all*, también es necesaria una regla Ingress que autorice la conexión en los Pods de Nginx (regla 2 siguiente).
- Regla Ingress en los Pods de Nginx: permite recibir conexiones TCP en el puerto 80 desde el Pod de pruebas. No es necesaria una regla Egress adicional en los Pods de los servidores web para enviar las respuestas a esas conexiones, ya que el tráfico de retorno está permitido por el seguimiento del estado de las conexiones. No obstante, si se mantiene aplicada la política *default-deny-all*, también es necesaria una regla Egress que autorice el inicio de la conexión en el Pod de pruebas (regla 1 anterior).

3.4 - Exploración de otras funcionalidades de Terraform para Kubernetes

A demás de las funcionalidades trabajadas en esta práctica, se recomienda explorar otras posibilidades de Terraform para definir y gestionar recursos de Kubernetes mediante el lenguaje HCL. Estas funcionalidades pueden resultar de interés para realizar despliegues más completos y flexibles, y algunas de ellas se utilizarán en el trabajo final de la asignatura. Se plantean las siguientes líneas de exploración:

- Uso de ConfigMaps y Secrets. Investigar cómo definir estos recursos mediante Terraform para gestionar datos de configuración e información sensible. Analizar cómo utilizarlos desde los Pods para proporcionar variables de entorno o ficheros de configuración de arranque de los contenedores.
- Uso de volúmenes y montaje de ficheros. Explorar cómo definir volúmenes y montarlos en los contenedores para proporcionar ficheros de configuración, utilizando, por ejemplo, *ConfigMaps*, *Secrets* o volúmenes persistentes cuando sea necesario.
- Despliegue de recursos *Ingress*. Investigar cómo definir recursos Ingress mediante Terraform para configurar el acceso HTTP y HTTPS a las aplicaciones desplegadas en el clúster mediante URLs amigables, utilizando el controlador Ingress ya instalado en el clúster de Kubernetes.

- Definición de recursos *NetworkAttachmentDefinition*. Explorar cómo desplegar mediante Terraform recursos *NetworkAttachmentDefinition* para configurar redes adicionales a las que puedan conectarse los Pods mediante Multus.
- Configuración del arranque e inicialización de los contenedores. Analizar el uso de los campos *command* y *args* en las definiciones de los contenedores de los Pods, campos que permiten especificar o modificar el comando de arranque y sus argumentos. Explorar también el uso de *initContainers,* que permiten ejecutar tareas de inicialización antes de arrancar los contenedores principales del Pod, por ejemplo, para generar ficheros de configuración, preparar directorios, copiar archivos o comprobar que determinados servicios están disponibles. Investigar cómo definir estas funcionalidades mediante Terraform y cómo utilizarlas para adaptar el despliegue de las aplicaciones al entorno de ejecución.

Para explorar estas funcionalidades, se recomienda consultar la documentación oficial de Terraform para Kubernetes, donde se describe la sintaxis HCL de los recursos disponibles y se incluyen ejemplos de configuración [9]. Además, en la carpeta de ejemplos del repositorio GitHub oficial *"Terraform Kubernetes Provider"* [10] puede encontrar algunos ejemplos adicionales.

Cuando se necesite desplegar un recurso de Kubernetes que no disponga de un recurso específico en el proveedor, también puede estudiarse el uso de *kubernetes_manifest,* que permite definir recursos a partir de su representación en HCL, incluidos recursos personalizados como los *NetworkAttachmentDefinition*. Para más información sobre el uso de *kubernetes_manifest* consulte la información disponible en [11].

Por último, se recomienda considerar la posibilidad de configuración de reglas de seguridad mediante recursos *NetworkPolicy* para controlar el tráfico de acceso permitido hacia los Pods desde el exterior del clúster de Kubernetes. Su configuración y utilidad se concretará en el trabajo final de la asignatura.

Referencias

1 Terraform Cookbook. https://learning.oreilly.com/library/view/terraform-cookbook/9781098108458/

2 Terraform: Up and Running, 3rd Edition. https://learning.oreilly.com/library/view/terraform-up-and/9781098116736/

3 Terraform OpenStack Provider. https://registry.terraform.io/providers/terraform-provider-openstack/openstack/latest/docs

4 Terraform OpenStack Provider (GitHub Repository). https://github.com/terraform-provider-openstack/terraform-provider-openstack

5 Infomaniak OpenStack Public Cloud Guide - Getting Started with Terraform. https://docs.infomaniak.cloud/documentation/tutorials/08.terraform/01.getting-started/

6 Terraform OpenStack: Automating Application Server Scaling and Management. https://www.cloudbolt.io/eguides/data-center-automation/terraform-openstack/

7 Tutorial: Infraestructura como código con Terraform. https://galvarado.com.mx/post/tutorial-infraestructura-como-c%C3%B3digo-con-terraform/

8 Calico CNI. https://docs.tigera.io/calico/latest/about/

9 Terraform Kubernetes Provider. https://registry.terraform.io/providers/hashicorp/kubernetes/latest/docs

10 Terraform Kubernetes Provider (GitHub Repository). https://github.com/hashicorp/terraform-provider-kubernetes

11 Terraform Kubernetes Provider: kubernetes_manifest. https://registry.terraform.io/providers/hashicorp/kubernetes/latest/docs/resources/manifest

12 Manage Kubernetes resources with Terraform. https://developer.hashicorp.com/terraform/tutorials/kubernetes/kubernetes-provider

13 Terraform Kubernetes Provider: Manage & Deploy Resources. https://spacelift.io/blog/terraform-kubernetes-provider

14 How to Manage Kubernetes Resources with the Terraform Kubernetes Provider. https://oneuptime.com/blog/post/2026-02-20-terraform-kubernetes-provider/view

15 Using Terraform with Kubernetes: A Comprehensive Guide. https://medium.com/@vinoji2005/using-terraform-with-kubernetes-a-comprehensive-guide-237f6bbb0586